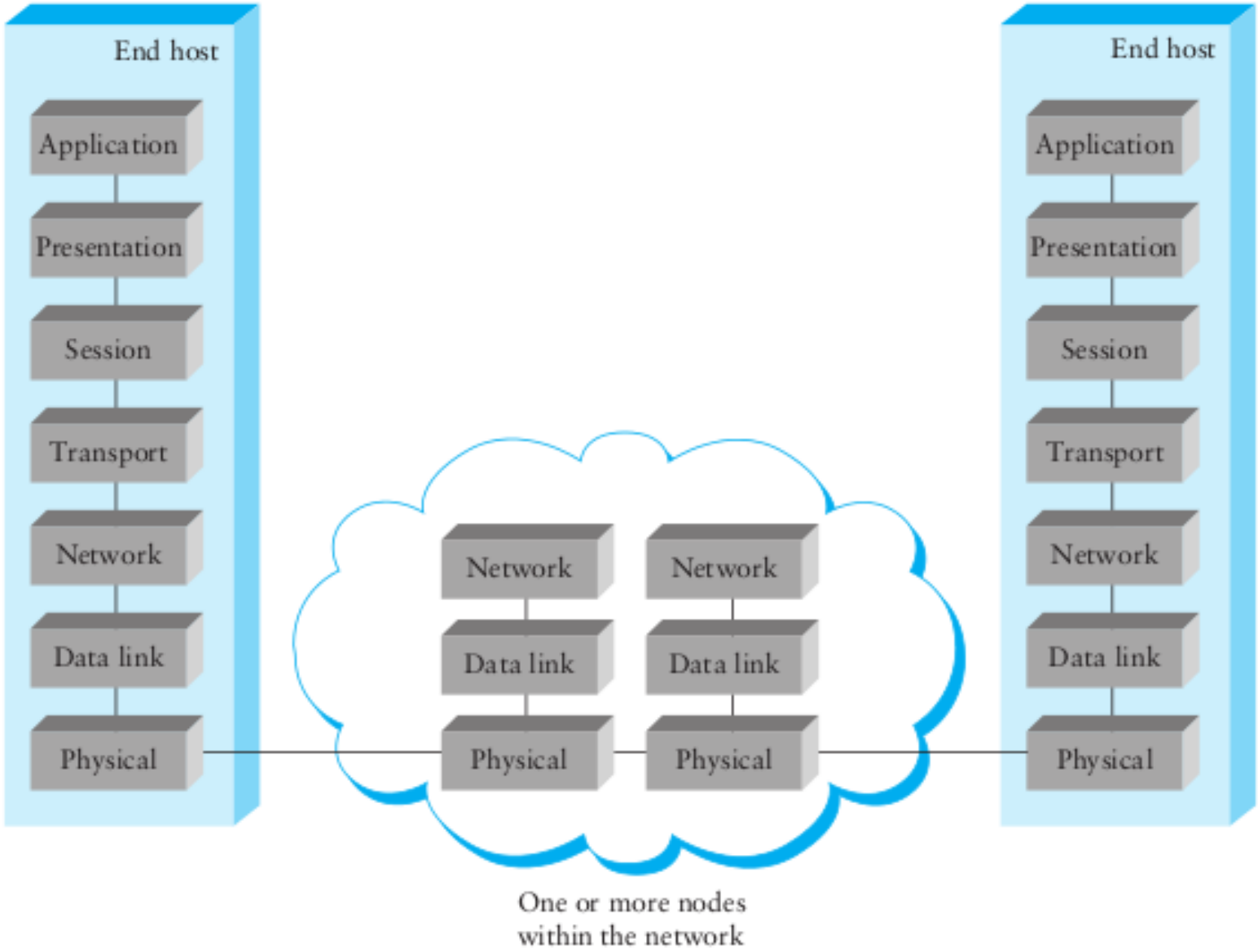
Capa de Aplicación

DC - FCEyN - UBA

Segundo cuatrimestre 2026

Arquitectura en capas



Menú del día

Capa de Aplicación

- **Correo Electrónico:** SMTP, MIME, POP3 e IMAP
- **Web:** HTTP 1.0, HTTP 1.1, Técnicas de Cacheo
- **Integración y Ejercicios**

Utilización de protocolos existentes

Específicos de la capa de aplicación

- Aplicaciones son software (procesos)
- Tipo de servicio de las capas inferiores.
- Arquitecturas de comunicación: Paradigma cliente - servidor, Paradigma P2P, Híbridos, RPC
- Syntaxis y semántica de los mensajes.

Arquitectura de las aplicaciones tradicionales

De tipo Request / Reply de mensajes entre cliente y servidor.

Aplicación versus protocolo

Ejemplo: HTTP (Protocolo) $\neq$ Firefox (Aplicación).

SMTP, POP3, IMAP, Webmail, MIME

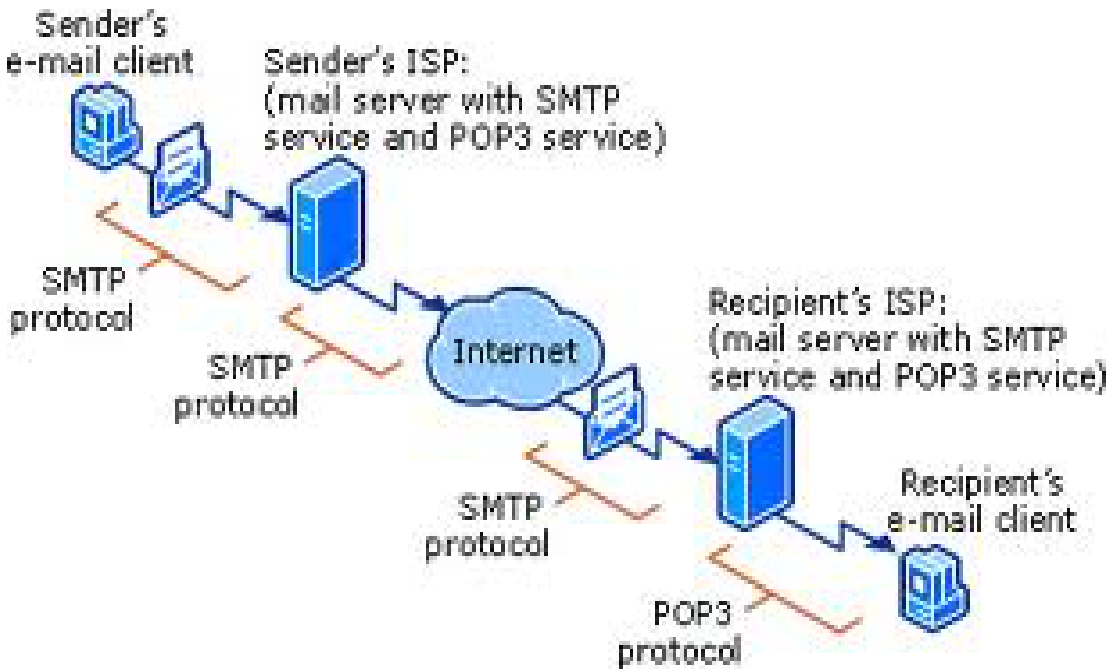




Infraestructura

Actores

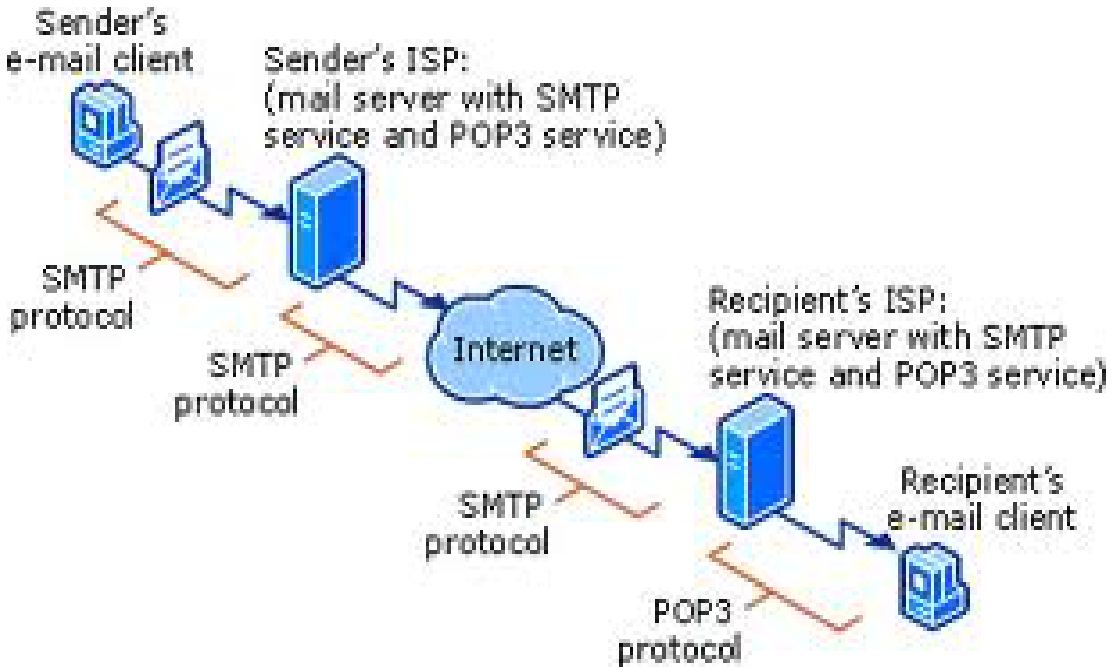
- Servidor saliente
- Servidor entratnte
- Relay
- Interfaz de usuario



Infraestructura

Protocolos

- Servidor saliente: SMTP (RFC 2821)
- Servidor entratnte: POP3 (RFC 1939)/IMAP (RFC 3501)
- Relay
- Interfaz de usuario: User Agent



SMTP: Mensajes

Comandos principales del Cliente (RFC 5321)

- `HELO` / `EHLO`: Identifica al cliente ante el servidor.
- `MAIL FROM: <remitente>`: Especifica el emisor del sobre (*envelope*).
- `RCPT TO: <destinatario>`: Especifica el receptor (puede repetirse).
- `DATA`: Inicia el envío de cabeceras y cuerpo; finaliza con `<CRLF>.<CRLF>`.
- `RSET`: Cancela la transacción actual y reinicia el estado.
- `QUIT`: Cierra la sesión y la conexión TCP.

Códigos de respuesta del Servidor

- `2xx` (Éxito): ej. 220 (Servicio listo), 250 (Acción completada OK).
- `3xx` (Intermedio / Esperando): ej. 354 (Comience envío de datos).
- `4xx` (Error temporal): ej. 421 (Servicio no disponible temporalmente).
- `5xx` (Error permanente): ej. 550 (Buzón de destino inexistente).

SMTP: Ejemplo de Sesión

Intercambio Cliente (C) / Servidor (S) en el puerto 25

```
S: 220 mail.midominio.com ESMTP listo
C: HELO miequipo.midominio.com
S: 250 Hello, pleased to meet you
C: MAIL FROM: <yo@midominio.com>
S: 250 Ok
C: RCPT TO: <destinatario@sudominio.com>
S: 250 Ok
C: DATA
S: 354 Start mail input; end with <CRLF>.<CRLF>
C: Subject: Consulta de clase
C: From: yo@midominio.com
C: To: destinatario@sudominio.com
C: Hola, esto es una prueba.
C: Hasta luego.
C: .
S: 250 Ok: queued as 12345
C: QUIT
S: 221 Bye
```

Mensaje de correo

Formato de mensaje estándar

- Header (Comandos Separados por CRLF).
  - From:.
  - To:, Cc:, etc.
- Body (ASCII plano).

Extensiones al formato: MIME

- En los mensajes se transfiere solo texto en formato ASCII.
- La interpretación de mensajes queda del lado del RFC822 y las extensiones MIME (RFC 2045).

Extensiones al formato de mensaje

Multipurpose Internet Mail Extensions

- ¿Por qué?
  - Quiero poder mandar más que solo texto plano.
- ¿Qué permite?
  - Transferir contenido multimedia sin modificar los protocolos básicos.
- ¿Qué se agrega para poder implementarlo?
  - Nuevos campos en el header (MIME-Version:, Content-Type:).
  - Definiciones de tipos y subtipos (Imagenes, Videos, Documentos).
- ¿Cómo hago que todo sea ASCII?
  - Encodings para codificar los distintos tipos de datos en ASCII (base64).
- ¿Cómo hago para enviar más de un tipo de archivo?
  - Uso delimitadores como tecnica de framing.

Ejemplo: Mensaje MIME Multipart

```
MIME-Version: 1.0
Content-Type: multipart/mixed;
boundary="--------417CA6E2DE4ABCAFBC5"
From: Alice Smith <Alice@cisco.com>
To: Bob@cs.Princeton.edu
Subject: promised material
Date: Mon, 07 Sep 1998 19:45:19 -0400

----------417CA6E2DE4ABCAFBC5
Content-Type: text/plain; charset=us-ascii
Content-Transfer-Encoding: 7bit

Bob,

Here's the jpeg image and draft report I promised.

--Alice

----------417CA6E2DE4ABCAFBC5
Content-Type: image/jpeg
Content-Transfer-Encoding: base64
... unreadable encoding of a jpeg figure
```

POP3: Post Office Protocol (RFC 1939)

Características

- Usa TCP en el puerto 110.
- Paradigma *Download and Delete* (orientado al cliente): los correos se descargan y gestionan localmente.
- Simple y liviano; no sincroniza carpetas ni estado (leído/no leído) entre múltiples dispositivos.

Fases de la Sesión

- **Autorización:** El cliente se identifica y autentica (`USER`, `PASS`).
- **Transacción:** El cliente consulta y descarga correos (`STAT`, `LIST`, `RETR`) o los marca para borrar (`DELE`).
- **Actualización:** Al enviar `QUIT`, el servidor elimina definitivamente los mensajes marcados y libera el buzón.

POP3: Mensajes

Comandos principales del Cliente (RFC 1939)

- `USER <usuario>` / `PASS <clave>`: Autenticación de credenciales.
- `STAT`: Retorna cantidad de mensajes y tamaño total en octetos.
- `LIST [n]`: Lista identificadores de mensajes y sus tamaños.
- `RETR n`: Solicita la transmisión del mensaje número *n*.
- `DELE n`: Marca el mensaje *n* para eliminación al cerrar sesión.
- `RSET`: Desmarca los mensajes pendientes de eliminación.
- `QUIT`: Aplica borrados definitivos (fase UPDATE) y cierra conexión.

Respuestas del Servidor

- `+OK [comentario]`: Éxito en el procesamiento del comando.
- `-ERR [comentario]`: Error o fallo en la ejecución solicitada.

POP3: Ejemplo de Sesión

Intercambio Cliente (C) / Servidor (S) en el puerto 110

```
S: +OK POP3 server ready <mail.servidor.com>
C: USER alice
S: +OK User accepted
C: PASS secret123
S: +OK Mailbox open, 2 messages (320 octets)
C: STAT
S: +OK 2 320
C: LIST
S: +OK 2 messages (320 octets)
S: 1 120
S: 2 200
S: .
C: RETR 1
S: +OK 120 octets
S: <el servidor envía el contenido del mensaje 1>
S: .
C: DELE 1
S: +OK message 1 deleted
C: QUIT
S: +OK POP3 server signing off (messages deleted)
```

IMAP: Internet Message Access Protocol (RFC 3501)

Características

- Usa TCP en el puerto 143.
- Paradigma *Keep and Sync*: los correos y carpetas permanecen en el servidor de forma remota y persistente.
- Soporta múltiples carpetas/buzones jerárquicos y búsquedas en el servidor.
- Permite el acceso simultáneo y sincronizado desde múltiples dispositivos.

Acceso por Partes y Estado

- **Descarga selectiva:** Permite obtener solo cabeceras, estructura MIME o adjuntos específicos sin bajar el correo entero.
- **Flags de estado:** Mantiene marcas sincronizadas en el servidor (`\Seen`, `\Answered`, `\Flagged`, `\Deleted`).

IMAP: Comandos del Cliente (RFC 3501)

Comandos de Sesión y Buzones

- `LOGIN <user> <pass>`: Autentica al cliente ante el servidor.
- `LIST <ref> <mailbox>`: Lista carpetas y buzones disponibles.
- `SELECT <mailbox>`: Abre un buzón para lectura/escritura (`EXAMINE` en solo lectura).
- `LOGOUT`: Cierra la sesión y finaliza la conexión TCP.

Comandos de Mensajes

- `FETCH <id> <items>`: Descarga selectiva de atributos, cabeceras o partes MIME de un correo.
- `STORE <id> <flags>`: Modifica marcas de estado (ej. agregar flag `\Deleted`).
- `EXPUNGE`: Elimina definitivamente todos los mensajes marcados como borrados.

IMAP: Sintaxis y Respuestas del Servidor

Identificadores del Cliente (Tags: a1, a2, ...)

- Prefijo alfanumérico generado por el cliente en cada comando.
- Permite enviar múltiples comandos en simultáneo de forma asincrónica (*pipelining*) y asociar inequívocamente cada respuesta con su solicitud.

Tipos de Respuestas del Servidor

- **Etiquetadas (*Tagged*):** Indican el resultado final del comando asociado a ese tag:
  - `OK`: El comando se completó exitosamente.
  - `NO`: Fallo operacional (ej. buzón no encontrado, clave inválida).
  - `BAD`: Error de protocolo o de sintaxis en el comando.
- **No etiquetadas (*Untagged*, con `*`):** Envían información intermedia solicitada (ej. datos de `FETCH` o carpetas de `LIST`) o eventos espontáneos del servidor (ej. avisos de nuevos correos).

IMAP: Ejemplo de Sesión

Intercambio Cliente (C) / Servidor (S) en el puerto 143

```
S: * OK IMAP4rev1 Service Ready
C: a1 LOGIN alice secreto123
S: a1 OK LOGIN completed
C: a2 LIST  "*"  % Lista carpetas
S: * LIST (\HasNoChildren) /INBOX"
S: a2 OK LIST completed
C: a3 SELECT INBOX  % Selecciona buzón
S: * 2 EXISTS
S: * OK [UIDVALIDITY 3857529] UIDs valid
S: a3 OK [READ-WRITE] SELECT completed
C: a4 FETCH 1 BODY[HEADER.FIELDS (FROM SUBJECT)]
S: * 1 FETCH (BODY[...] 64
S:   From: bob@ejemplo.com
S:   Subject: Consulta de clase)
S: a4 OK FETCH completed
C: a5 LOGOUT
S: * BYE Logging out
S: a5 OK LOGOUT completed
```

Conceptos fundamentales

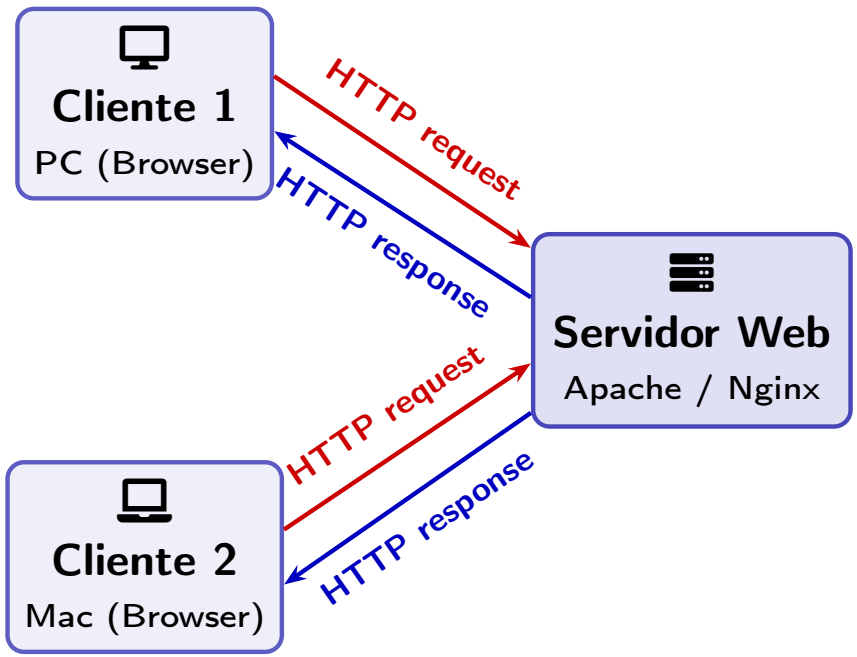
Generales

- Usa TCP (puerto 80).
- Browser y servidor se intercambian mensajes
- HTTP $\Rightarrow$ HEADER
- HTML $\Rightarrow$ BODY.
- Cada objeto se referencia por un identificador único (URL).
- HTTP **no** mantiene estado.

HTTP: Arquitectura y Funcionamiento

Hypertext Transfer Protocol

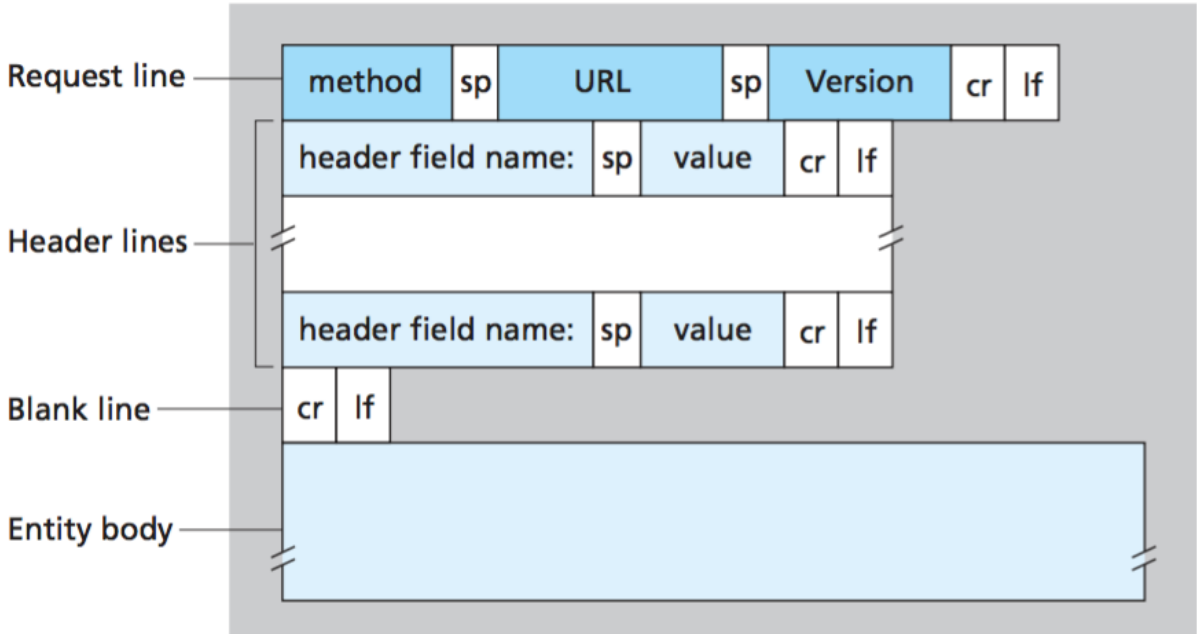
- Protocolo de capa de aplicación de la Web.
- **Modelo cliente / servidor**:
  - **Cliente:** browser que requiere, recibe y despliega recursos Web.
  - **Servidor:** servidor Web que envía recursos en respuesta a peticiones.
- **Estándares**:
  - HTTP 1.0: RFC 1945
  - HTTP 1.1: RFC 2616 / RFC 7230



Petición HTTP

Tipos

- GET: Solicitar información
- HEAD: Solicitar encabezados
- POST: Enviar datos para crear recurso
- PUT: Enviar datos para actualizar/reemplazar recurso
- DELETE: Solicitar eliminación de recurso



Petición HTTP: Ejemplo de Estructura

Línea de requerimiento
(comandos `GET`, `POST`, `HEAD`)

Líneas de encabezado
Metadatos de la petición

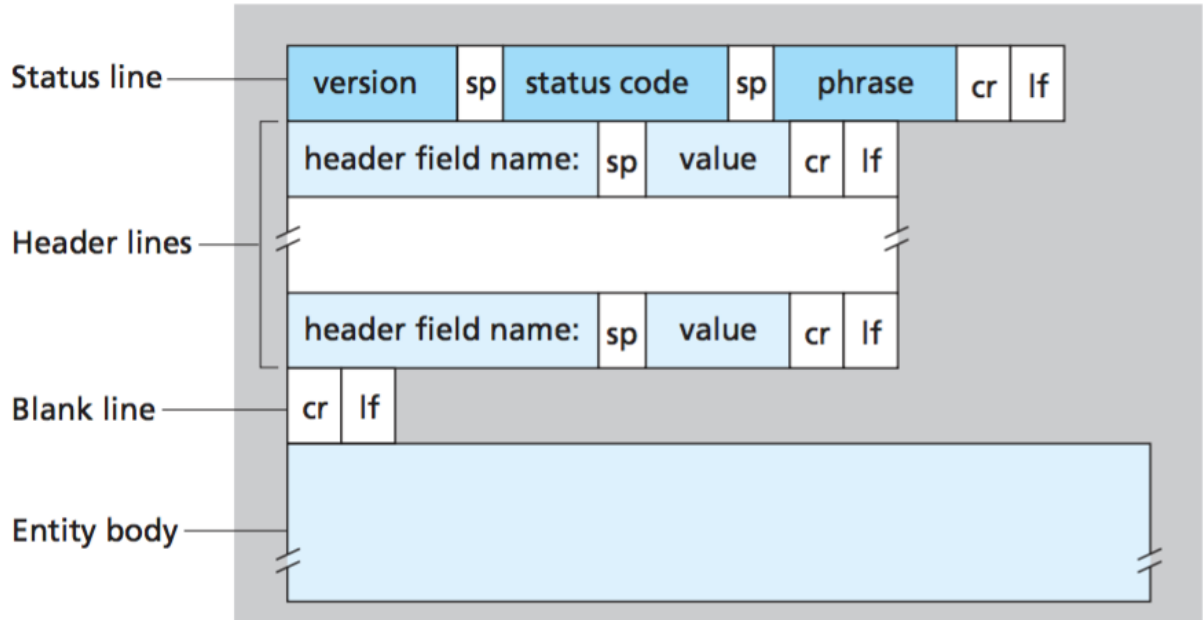
Carriage return, line feed
Indica fin del encabezado

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
User-Agent: Mozilla/4.0
Connection: close
Accept-Language: es
⟨carriage return, line feed extra⟩
```

Respuesta HTTP

Tipos

- 1xx. Informativo
- 2xx. Exitoso
- 3xx. Redireccion
- 4xx. Error del Cliente
- 5xx. Error del Servidor



Respuesta HTTP: Ejemplo de Estructura

Línea de estatus
Código y frase de estatus del protocolo

Líneas de encabezado
Metadatos de la respuesta

data (archivo solicitado)
Recurso transferido (ej. documento HTML)

```
HTTP/1.1 200 OK
Connection: close
Date: Thu, 06 Aug 1998 12:00:15 GMT
Server: Apache/1.3.0 (Unix)
Last-Modified: Mon, 22 Jun 1998 ...
Content-Length: 6821
Content-Type: text/html
data data data data data ...
```

Encabezados HTTP Esenciales

Encabezado	Petición/Respuesta	Contenido
`Host`	Petición	Nombre de dominio del servidor solicitado.
`Content-Type`	Ambos	Tipo de medio (ej. HTML, JSON, Imagen) de los datos.
`Content-Length`	Ambos	Tamaño del cuerpo del mensaje en bytes.
`Cache-Control`	Ambos	Directivas para el control de la caché.
`If-Modified-Since`	Petición	Se solicita el recurso si fue modificado despues de la fecha.
`Authorization`	Petición	Credenciales de autenticación del cliente.
`Cookie`	Petición	Envía cookies almacenadas del cliente al servidor.
`User-Agent`	Petición	Identifica el software del agente de usuario.

Ejercicio 1

El protocolo HTTP permite hacer distintos tipos de pedidos para recursos de un determinado dominio. Escriba los Requests HTTP 1.1 que permitan obtener los siguientes pedidos al sitio web del departamento de computación:

1. El recurso /
2. Encabezado del recurso `/tdc`
3. El recurso `/logo.jpg` si no fue modificado desde una determinada fecha.
4. Crear un nuevo post en el foro del departamento

Resolución: Ejercicio 1 (1/2)

1. Recurso raíz /

```
GET / HTTP/1.1
Host: www.dc.uba.ar
User-Agent: Mozilla/5.0
[Línea en blanco]
```

- `Host:` es obligatorio en HTTP/1.1 (permite *Virtual Hosting*).

2. Encabezado del recurso `/tdc`

```
HEAD /tdc HTTP/1.1
Host: www.dc.uba.ar
[Línea en blanco]
```

- Mismos encabezados que `GET`, pero el servidor **no envía cuerpo**.
- Permite chequear existencia, tipo o tamaño ahorrando ancho de banda.

Resolución: Ejercicio 1 (2/2)

3. Recurso /logo.jpg condicional

```
GET /logo.jpg HTTP/1.1
Host: www.dc.uba.ar
If-Modified-Since: Mon, 05 Oct 2026 12:00:00 GMT
```

- Si no cambió: responde `304 Not Modified` (sin cuerpo).
- Si cambió: responde `200 OK` con el archivo actualizado.

4. Crear un post en el foro

```
POST /foro/posts HTTP/1.1
Host: www.dc.uba.ar
Content-Type: application/x-www-form-urlencoded
Content-Length: 39

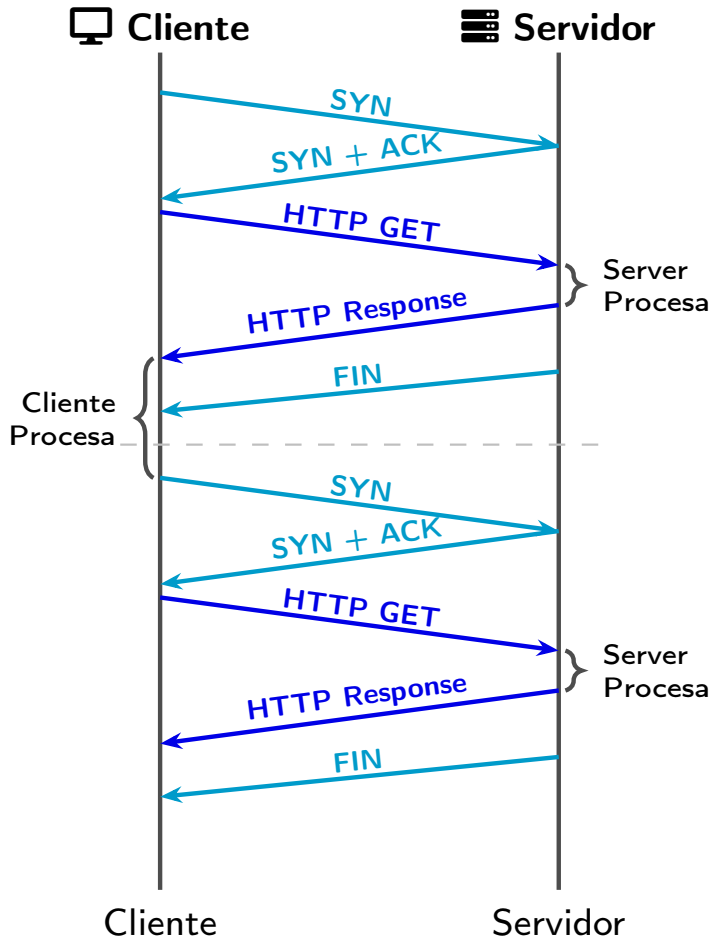
titulo=Consulta&cuerpo=Mensaje+del+foro
```

Acción no idempotente. Requiere `Content-Type` y `Content-Length`. Servidor suele responder `201 Created` (con `Location: ...`) o `200 OK`.

HTTP 1.0: Conexión No Persistente

Características

- Requiere una conexión TCP independiente por cada objeto.
- **Costo por objeto:**
  - $1 \times$ RTT (Handshake TCP).
  - $1 \times$ RTT (HTTP Request + Response).
- Total: $2 \times$ RTT por recurso + transmisión.
- Cierre con `FIN` tras cada respuesta.
- Sobrecarga de sockets y buffers en el SO.



HTTP 1.0: Cálculo de RTT y Cierre TCP

Cómputo del tiempo de transferencia ($2 \times$ RTT por objeto)

- $1 \times$ RTT **Handshake TCP:** `SYN` (ida) + `SYN + ACK` (vuelta). Conexión establecida.
- $1 \times$ RTT **HTTP Request / Response:** `HTTP GET` (ida, con `ACK` vía *piggybacking*) + `HTTP Response` (vuelta con datos).
- **Total:** $2 \times$ RTT por recurso (+ tiempo de transmisión en el servidor).

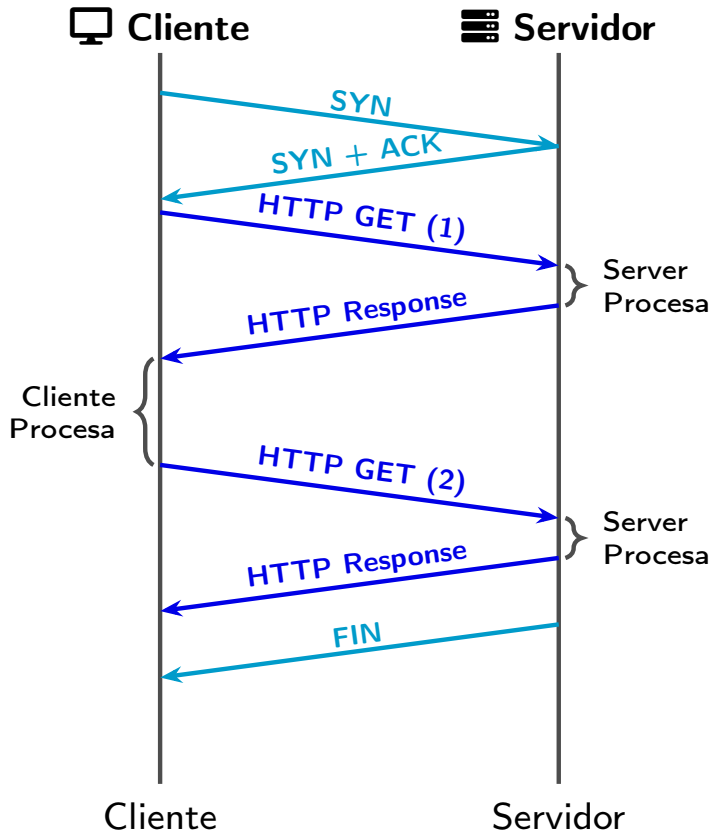
¿Por qué el cierre (`FIN` / `ACK`) no suma RTTs al cálculo?

- **Señal de fin de archivo (EOF):** El flag `FIN` viaja típicamente en el último segmento de datos o inmediatamente tras él (sin requerir RTT adicional previo).
- **No bloquea la siguiente petición:** El cliente no espera el cierre de 4 vías ni `TIME_WAIT`; abre de inmediato otro socket con un nuevo puerto efímero e inicia el próximo `SYN`.

HTTP 1.1: Conexión Persistente (sin Pipelining)

Características

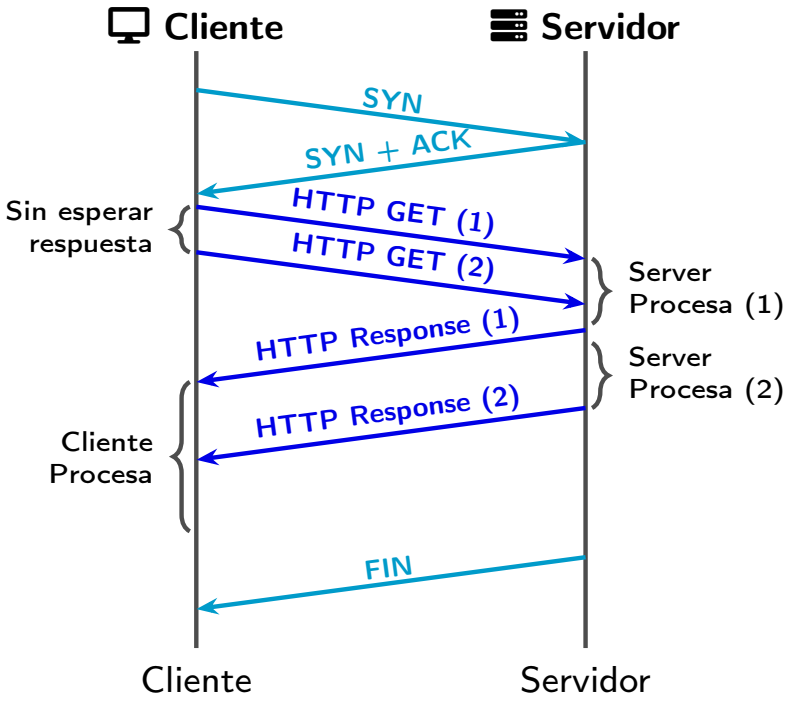
- Reutiliza la conexión TCP para múltiples recursos.
- Solo 1 × RTT inicial de Handshake TCP.
- Cada recurso siguiente cuesta solo 1 × RTT (sin nuevo handshake).
- Evita sobrecarga de apertura y cierre reiterado en el SO.
- Menor latencia total de carga de página.



HTTP 1.1: Conexión Persistente (con Pipelining)

Características

- Reutiliza la conexión TCP para múltiples recursos.
- Envía peticiones en ráfaga **sin esperar** cada respuesta.
- Costo inicial: $1 \times$ RTT (Handshake TCP).
- Costo objetos: $\approx 1 \times$ RTT para todas las peticiones en paralelo.
- Respuestas en orden estricto (FIFO).
- Riesgo de *Head-of-Line (HoL) Blocking*.



Etiquetas HTML Relevantes

Algunas etiquetas HTML provocan que el navegador realice solicitudes HTTP para obtener recursos:

Recursos embebidos (descarga automática)

- `<script>`: Carga scripts externos (atributo `src`).
- `<link rel="stylesheet">`: Carga hojas de estilo CSS externas (atributo `href`).
- `<img>`, `<audio>`, `<video>`: Carga archivos multimedia (atributo `src`).

Acciones del usuario (navegación e interacción)

- `<form>`: Envía petición con datos al servidor (**al enviar**).
- `<a>`: Solicita un nuevo recurso vía `href` (**al hacer clic**).

Ejercicio 2

Suponga la siguiente página escrita en HTML que reside en el servidor `www.fcen.uba.ar`:

```
1  <html>
2     <head>
3        <title>Facultad de Ciencias Exactas y Naturales</title>
4        <link rel="stylesheet" type="text/css" href="style.css" />
5     </head>
6     <body>
7        <div>
8           <img src="searchline.png" />
9           <a href="avsearch.php"> <img src="home.png" /> </a>
10       </div>
11       <div>
12          <form name="searchform" action="search">
13             <label>Buscar</label>
14             <input name="SearchableText" type="text" title="Buscar en el Sitio" />
15             <input type="image" src="search_icon.gif" />
16          </form>
17       </div>
18    </body>
19 </html>
```

1. ¿Cuánto tiempo en términos de RTTs transcurrirá como mínimo, hasta transferir la totalidad de la información en HTTP/1.0?
2. ¿Y en HTTP/1.1?

Resolución: Ejercicio 2 - Objetos a Descargar

Objetos identificados en el HTML

Al procesar el HTML, el navegador solicita automáticamente:

1. **Recurso base**: el documento HTML inicial.
2. Línea 4: `<link rel="stylesheet" href="style.css">` (`style.css`).
3. Línea 8: `<img src="searchline.png">` (`searchline.png`).
4. Línea 9: `<img src="home.png">` dentro del link (`home.png`).
5. Línea 15: `<input type="image" src="search_icon.gif">` (`search_icon.gif`).

Total: 1 HTML base + 4 objetos embebidos = 5 objetos.

Recursos que NO se descargan automáticamente

- `<a href="avsearch.php">`: Requiere clic del usuario.
- `<form action="search">`: Requiere envío (*submit*) del usuario.

Resolución: Ejercicio 2 - Cálculo de RTTs

1. HTTP/1.0 (Conexión No Persistente, Secuencial)

Cada objeto requiere su propia conexión TCP independiente:

- **Costo por objeto:**
  1 RTT (Handshake TCP) + 1 RTT (HTTP GET) = 2 RTT.
- **HTML base:** 1×2 RTT $= 2$ RTT.
- **4 objetos referenciados:** 4×2 RTT $= 8$ RTT.

Total HTTP/1.0 $= 2$ RTT $+ 8$ RTT $= 10$ RTT (5 objetos $\times$ 2 RTT).

2. HTTP/1.1 (Conexión Persistente)

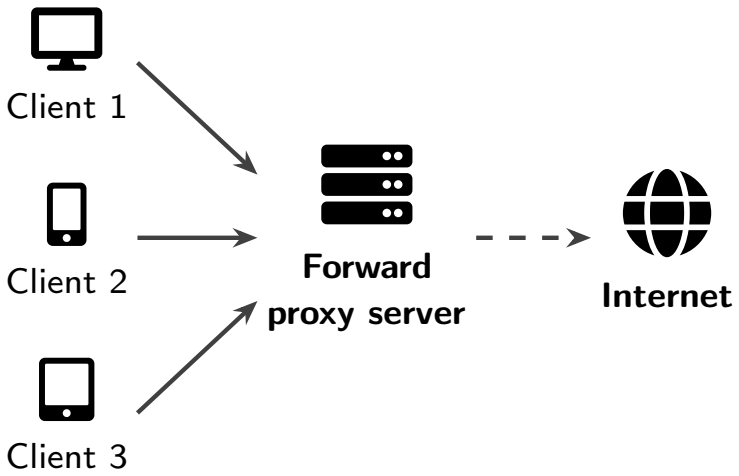
Reutiliza la conexión TCP inicial del HTML base (2 RTT):

- **Con Pipelining (en ráfaga):**
  2 RTT (HTML) + 1 RTT (4 objetos en paralelo) = 3 RTT.
- **Sin Pipelining (secuencial sobre la misma conexión):**
  2 RTT (HTML) + 4×1 RTT (HTTP) = 6 RTT.

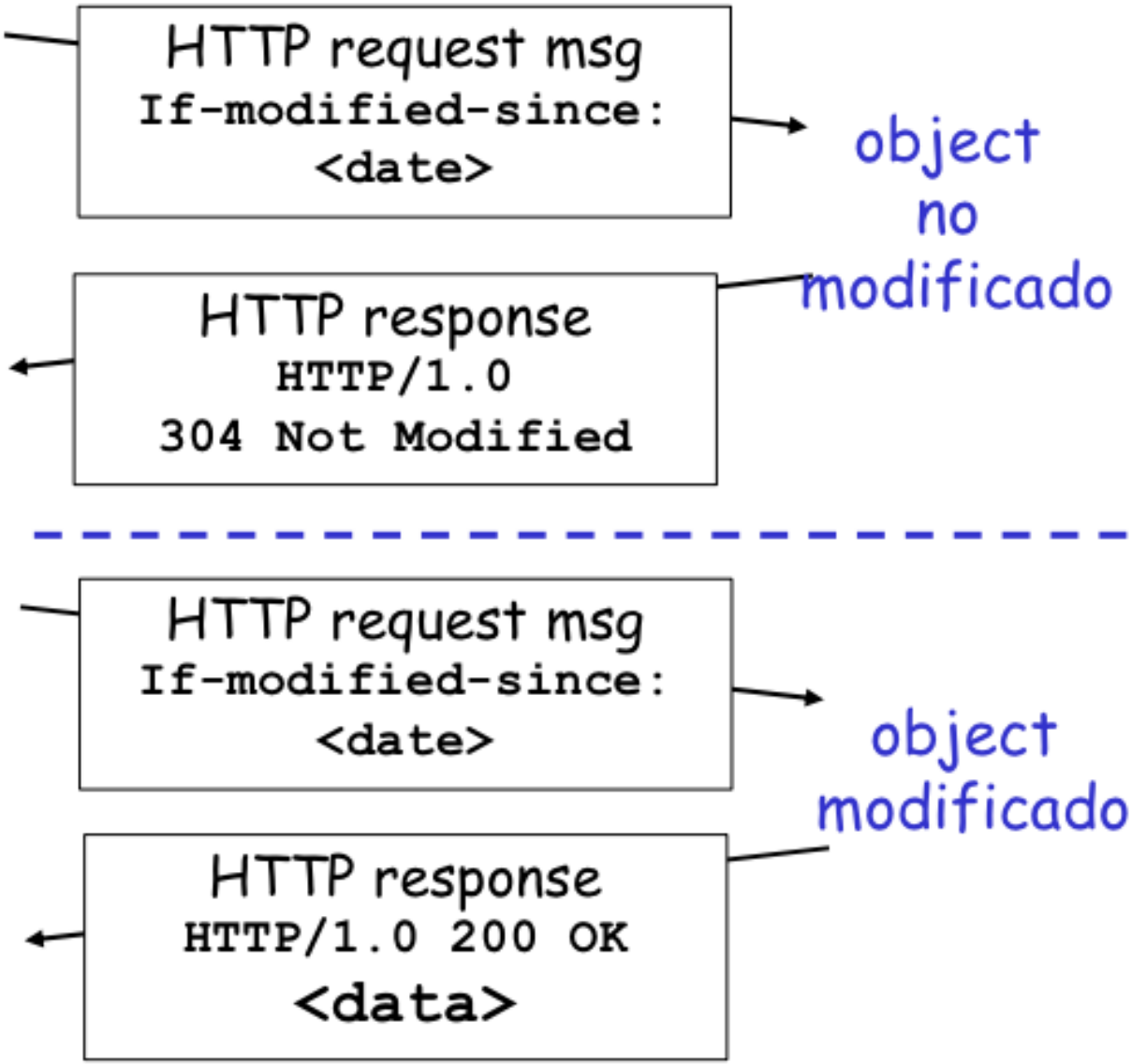
Técnicas de cacheo: Proxy Cache

Funcionamiento del Proxy

- El cliente configura su navegador para redirigir peticiones al **Proxy**.
- El Proxy intercepta las solicitudes:
  - **Cache Hit:** responde directamente con el recurso guardado.
  - **Cache Miss:** solicita el objeto al servidor web, lo guarda en caché y lo envía al cliente.
- **Ventajas:** menor tiempo de respuesta y ahorro de ancho de banda WAN.



Cacheo HTTP y formato de respuesta



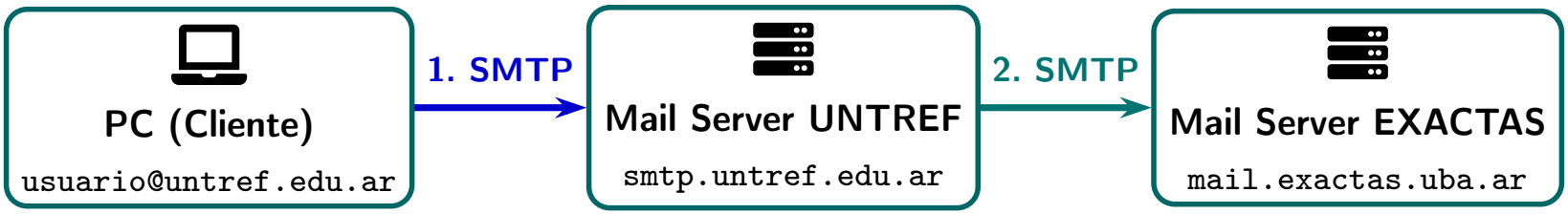
Ejercicio de Integración

Desde una PC en el dominio `untref.edu.ar` se envía un correo a `pedro@exactas.uba.ar` y luego se accede al recurso web `http://www.exactas.uba.ar/index.html`. La PC se conecta a la web a través de un servidor proxy con dirección IP `208.190.1.22`.

a. Enumere los mensajes SMTP que son necesarios para el envío de un mail a `pedro@exactas.uba.ar` desde un host en el dominio `untref.edu.ar` hasta que llega al servidor de destino.

b. Detalle los mensajes HTTP (*Requests* y *Responses*) desde que la PC envía el primer GET hasta que le llega el respectivo *Response*.

Resolución: Arquitectura de Envío (Punto a)



Resolución: Mensajes SMTP (Punto a)

Salto 1: PC → Saliente (UNTREF)

```
S: 220 smtp.untref.edu.ar ESMTP ready
C: HELO host.untref.edu.ar
S: 250 Hello host.untref.edu.ar
C: MAIL FROM: <usuario@untref.edu.ar>
S: 250 Ok
C: RCPT TO: <pedro@exactas.uba.ar>
S: 250 Ok
C: DATA
S: 354 Start mail input
C: From: usuario@untref.edu.ar
C: To: pedro@exactas.uba.ar
C: Subject: Consulta de materia
C: Hola Pedro...
C: .
S: 250 Ok: queued
C: QUIT
S: 221 Bye
```

Salto 2: Saliente → Entrante (EXACTAS)

```
S: 220 mail.exactas.uba.ar ESMTP ready
C: HELO smtp.untref.edu.ar
S: 250 Hello smtp.untref.edu.ar
C: MAIL FROM: <usuario@untref.edu.ar>
S: 250 Ok
C: RCPT TO: <pedro@exactas.uba.ar>
S: 250 Ok
C: DATA
S: 354 Start mail input
C: From: usuario@untref.edu.ar
C: To: pedro@exactas.uba.ar
C: Subject: Consulta de materia
C: Hola Pedro...
C: .
S: 250 Ok: queued
C: QUIT
S: 221 Bye
```

Resolución: Integración (b. HTTP vía Proxy)

1. PC → Proxy (208.190.1.22)

```
GET /index.html HTTP/1.1
Host: www.exactas.uba.ar
```

2. Proxy → Servidor Origen (en caso de Cache Miss)

```
GET /index.html HTTP/1.1
Host: www.exactas.uba.ar
Via: 1.1 208.190.1.22
```

3. Respuestas (Servidor → Proxy → PC)

- Servidor responde 200 `OK` con el body HTML.
- Proxy cachea la respuesta y la reenvía (200 `OK`) a la PC.

¿Repasando...?

¿Dudas?,

¿Preguntas?